

C++ Space Telemetry Ground Station

Technical dossier for software recruiters

English version

C++20 portfolio project - Linux - telemetry - networking - multithreading -
CMake - testing - CI/CD

Document purpose

Present the project to a recruiter or software engineer by explaining the implementation goals, architecture, development choices, code quality practices and skills demonstrated by the codebase.

Author

Guillaume Lemonnier

Date

July 2026

Contents

1 Executive summary	2
2 Implementation intent	2
2.1 Demonstrate practical modern C++	2
2.2 Move the project toward an industrial context	2
2.3 Produce an interview-ready project	2
3 Delivered functional scope	3
4 Overall architecture	3
4.1 Code organization	3
5 Development choices	4
5.1 Why C++20	4
5.2 Why CMake	4
5.3 Error model	5
5.4 Strict binary protocol	5
5.5 UDP and TCP choices	5
6 Multithreaded pipeline	5
7 Degraded mode	6
7.1 Intent	6
7.2 Mechanism	6
8 Replay, export and observability	6
8.1 STGF replay format	6
8.2 CSV and JSON exports	6
8.3 Logs	6
9 Testing, quality and CI/CD	7
9.1 Testing strategy	7
9.2 GitHub Actions	7
10 Benchmark and performance	7
11 Skills demonstrated	7
12 Known limitations	8
13 Possible extensions	8
14 Interview pitch	8
15 Demo commands	9
15.1 Build and tests	9
15.2 Replay to CSV and JSON	9
15.3 UDP simulation with loss and corruption	9
15.4 Benchmark	9
16 Conclusion	9

1 Executive summary

C++ Space Telemetry Ground Station is a Linux-based C++20 project that simulates a ground station able to receive, decode, validate, replay and export satellite-like or embedded-device telemetry frames. It was designed as a skills demonstration for roles involving industrial software, systems programming, telemetry, networking, modern C++ and constrained environments.

The project covers an end-to-end chain: telemetry frame generation, UDP/TCP reception, TCP stream reconstruction, strict binary parsing, CRC-32 validation, a multi-threaded processing pipeline, replay files, CSV/JSON export, logging, automated tests, GitHub Actions CI, reproducible benchmarking and operational degraded mode.

Recruiter positioning

This project is not intended to be a certified space product. It demonstrates the ability to build a small but structured, testable, robust and explainable C++ system with development choices close to industrial, defense, space or embedded-software concerns.

2 Implementation intent

The project has three main goals.

2.1 Demonstrate practical modern C++

The goal was not to write a small syntax exercise, but a project where C++ is used to structure a real application pipeline: domain models, binary encoding, error handling, Linux I/O, thread-safe queues, tests and command-line tools.

The project exercises:

- C++20, RAII, strong domain types and modular design;
- `std::variant` to distinguish a valid frame from a parsing error;
- closeable thread-safe queues to decouple reception, decoding and writing;
- strict CMake builds, warnings and automated testing.

2.2 Move the project toward an industrial context

Telemetry was chosen deliberately because it introduces systems-oriented topics: binary protocols, integrity checks, network loss, data corruption, replay, state monitoring and performance measurement.

The project therefore includes:

- a strict binary protocol;
- CRC validation before trusting telemetry data;
- loss and corruption simulation;
- degraded mode when the incoming stream becomes too unstable;
- a benchmark to measure pipeline behavior.

2.3 Produce an interview-ready project

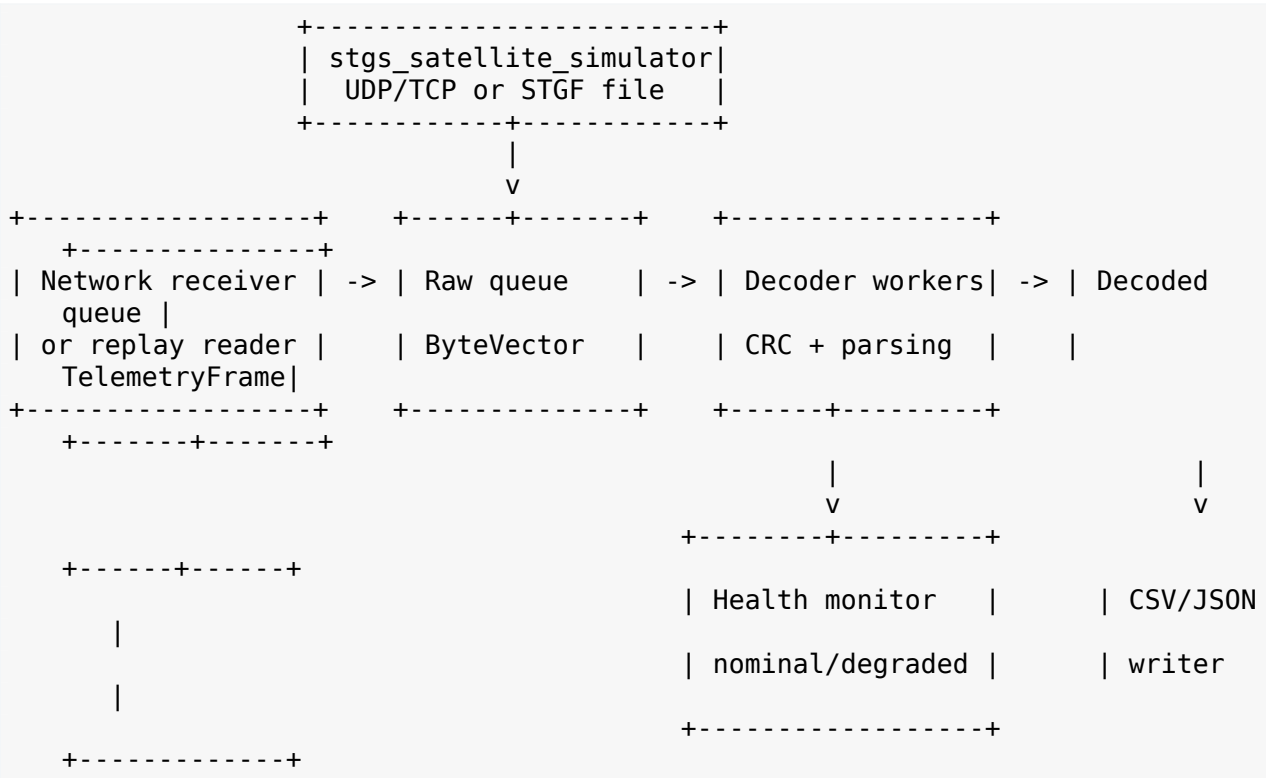
The codebase was organized to be easy to present: clear structure, technical documentation, English README, reproducible commands and separation between domain code, networking infrastructure, replay, tests and benchmarking.

3 Delivered functional scope

Version	Goal	Implemented features
V1	Core pipeline	Telemetry frame simulator, UDP receiver, C++ parser, CRC-32 validation, logs, unit tests, English README.
V2	Industrialization	Multithreaded pipeline, STGF replay files, CSV and JSON export, reproducible benchmark, CTest, GitHub Actions.
V3	Operational robustness	Stricter binary protocol, network loss and corruption simulation, degraded mode, architecture documentation, performance report.

4 Overall architecture

The architecture is deliberately split into simple components. The ground station can receive frames from the network or from a replay file. Raw frames are pushed into a queue, decoded by one or more workers, monitored by a health component and then exported.



4.1 Code organization

File or folder	Responsibility
----------------	----------------

apps/ground_station.cpp	Main application: CLI, UDP/TCP reception, replay, workers, exports and degraded mode.
apps/satellite_simulator.cpp	Telemetry generator: network output, STGF file output, loss, corruption and simulated rate.
include/stgs/TelemetryFrame.hpp	Domain model for a decoded telemetry frame.
include/stgs/FrameCodec.hpp	Encoding, decoding, validation and frame extraction from a TCP stream.
src/FrameCodec.cpp	
include/stgs/NetworkServer.hpp	UDP/TCP reception on Linux using POSIX sockets and poll().
src/NetworkServer.cpp	
include/stgs/BlockingQueue.hpp	Closeable producer/consumer queue for the multithreaded pipeline.
include/stgs/StationHealth.hpp	Operational health monitoring and nominal/degraded transitions.
src/StationHealth.cpp	
include/stgs/Replay.hpp	Reading and writing the STGF replay format.
src/Replay.cpp	
benchmarks/benchmark_decode.cpp	Reproducible benchmark for the decode pipeline.
tests/test_main.cpp	Unit tests and behavioral tests.
.github/workflows/ci.yml	Build, tests, smoke replay, JSON export validation and benchmark smoke.

5 Development choices

5.1 Why C++20

C++20 keeps the project close to systems and industrial software roles while providing a more expressive and safer style than legacy C++. The project favors explicit design: domain types, isolated functions, testable components and very few external dependencies.

Why few dependencies?

The project is intentionally lightweight. The goal is to show mastery of C++/Linux fundamentals rather than hide complexity behind frameworks. This also keeps the build simple in CI and makes the code easier to audit.

5.2 Why CMake

CMake structures the project into separate targets. The main targets are:

- stgs_core for core components;
- stgs_network for networking infrastructure;
- stgs_ground_station for the receiver application;
- stgs_satellite_simulator for the telemetry generator;
- stgs_tests for CTest;
- stgs_benchmark_decode for performance measurements.

This separation allows the core library to be built independently from the applications and reused by tests.

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build -j
ctest --test-dir build --output-on-failure
```

5.3 Error model

The parser returns a `std::variant<TelemetryFrame, FrameError>`. Invalid telemetry is treated as an expected data condition, not as an application exception.

This is appropriate for telemetry streams: in a noisy network, invalid data should not crash the process. It should be counted, logged and used as an input for degraded-mode decisions.

```
using FrameParseResult = std::variant<TelemetryFrame, FrameError>;

FrameParseResult decodeFrame(std::span<const std::uint8_t> bytes);
```

5.4 Strict binary protocol

The protocol contains a magic word, a version, a satellite identifier, a timestamp, measurements, a payload length and a CRC-32. All integers are encoded in big-endian order to match network byte order.

```
MAGIC | VERSION | SATELLITE_ID | TIMESTAMP_MS | TEMPERATURE_C |
BATTERY_PERCENT | STATUS | PAYLOAD_LEN | PAYLOAD | CRC32
```

The validation rules are strict: expected magic word, supported version, coherent length, maximum payload size, valid CRC, battery between 0 and 100 and known status.

Why a magic word?

With TCP, the transport is a byte stream and does not preserve application frame boundaries. The STGS magic word allows the receiver to resynchronize the stream if noise or corrupted bytes appear.

5.5 UDP and TCP choices

The project implements both approaches:

- in UDP mode, each datagram is treated as one complete candidate frame;
- in TCP mode, an extractor rebuilds frames from a byte stream.

This illustrates two networking models: simplicity and expected loss with UDP, transport reliability but application-level framing requirements with TCP.

6 Multithreaded pipeline

The pipeline separates responsibilities: reception, decoding, health monitoring and writing. The number of decoder workers is configurable with `--decoder-threads`. This design demonstrates concurrency without adding unnecessary complexity.

```
Receiver or replay reader
-> BlockingQueue<ByteVector>
    -> decoder worker 1
    -> decoder worker 2
    -> ...
    -> BlockingQueue<TelemetryFrame>
        -> CSV/JSON writer
```

The `BlockingQueue` component is closeable, which makes thread shutdown simpler. Once no more data is available, workers can exit cleanly without busy-waiting.

7 Degraded mode

7.1 Intent

Degraded mode was added deliberately to go beyond simple frame parsing. It introduces the idea of an operational ground-station state: even if the program keeps running, the telemetry stream can become poor enough to require a degraded-state signal.

7.2 Mechanism

StationHealthMonitor maintains a sliding window of recent samples. Each sample is either a rejected frame or a decoded frame. A decoded frame is considered critical if its status is CRITICAL or SAFE_MODE.

The station moves from NOMINAL to DEGRADED if:

- the rejection rate reaches the configured threshold;
- or the number of critical frames reaches the configured threshold.

It returns to NOMINAL when the rejection rate falls below the recovery threshold and the window no longer contains a problematic critical volume.

```
--degraded-window 100
--degraded-min-samples 20
--degraded-rejection-rate 0.10
--degraded-recovery-rate 0.03
--degraded-critical-count 3
```

Value for a recruiter

This part demonstrates the ability to think about operational behavior, not only nominal output. The code monitors errors, makes a state decision and logs transitions.

8 Replay, export and observability

8.1 STGF replay format

The STGF format records binary frames and replays them through the same pipeline as live network data. This matters for testing, reproducibility and incident analysis.

```
'S' 'T' 'G' 'F' 0 0 0 1
FRAME_LENGTH:uint32_be | FRAME_BYTES
```

8.2 CSV and JSON exports

CSV enables simple spreadsheet-style inspection. JSON supports integration with analysis tools, dashboards or automated checks. The format can be inferred from the extension or forced with --output-format.

```
./build/stgs_ground_station --replay frames.stgf --output replay.csv
./build/stgs_ground_station --replay frames.stgf --output replay.json
./build/stgs_ground_station --replay frames.stgf --output telemetry.out --
    output-format json
```

8.3 Logs

The logger is thread-safe and can write to the console or to a file. Logs help explain station behavior: startup, frame errors, health transitions and final summaries.

9 Testing, quality and CI/CD

9.1 Testing strategy

The tests cover critical behavior: known CRC vector, encode/decode round trip, rejection for bad magic, CRC mismatch, length mismatch, TCP extraction with noise, replay round trip, blocking queue closure and degraded-mode transitions.

```
cctest --test-dir build --output-on-failure
```

9.2 GitHub Actions

The CI workflow builds in Release mode, runs CTest, generates an STGF replay file, replays to CSV, replays to JSON, validates JSON with Python and runs a benchmark smoke test. The project is therefore designed to be verified automatically, not only run locally.

Quality choice

For a recruiter, CTest, CI, a benchmark and error-case tests are as important as the nominal feature set. They demonstrate maintainability and industrialization habits.

10 Benchmark and performance

The project includes `stgs_benchmark_decode`, a tool that measures decode-pipeline throughput without network or disk I/O. It pre-generates valid frames, pushes them into the queue, decodes them with a configurable number of workers and measures frames per second.

```
./build/stgs_benchmark_decode --frames 200000 --payload-size 64 --decoder-threads 4
```

The performance report documents the methodology and reference measurements. One useful observation is that multithreading does not automatically improve performance for small workloads: queue synchronization can cost more than the parsing work itself. This is a valuable interview point because it shows that concurrency choices should be measured.

11 Skills demonstrated

Skill	Evidence in the project
Modern C++	C++20, domain types, <code>std::variant</code> , RAI and library separation.
Linux	POSIX sockets, <code>poll()</code> , CLI tools, files, Linux build and execution.
Networking	UDP/TCP reception, TCP framing, loss and corruption simulation.
Binary protocol	Magic word, version, lengths, big-endian fields, CRC and variable payload.
Multithreading	Thread-safe queues, decoder workers, separate writer and clean shutdown.

Performance	Reproducible benchmark, frames/second measurements, payload/thread analysis.
CMake	Separate targets, build options for tests/benchmarks, CTest.
Testing	Unit tests and error-case tests, CTest and CI integration.
CI/CD	GitHub Actions with build, tests, replay, JSON validation and benchmark smoke.
Documentation	English README, architecture documentation and performance report.
Robustness	Invalid-frame rejection, health monitoring, degraded mode and logs.

12 Known limitations

The project is intentionally educational and portfolio-oriented. It does not claim to meet the requirements of a certified space system. Its main limitations are:

- telemetry-inspired protocol, but not CCSDS-compatible;
- no real hardware or driver layer;
- no long-term persistence or file rotation;
- no Prometheus metrics or web dashboard;
- no fuzzing or full network load testing;
- no system package or systemd deployment unit.

These limitations are useful to mention in interviews because they show the ability to distinguish a portfolio project from production-critical software.

13 Possible extensions

The most relevant next improvements would be:

- add a CCSDS-inspired header with a sequence counter;
- estimate UDP loss from frame sequence counters;
- add a WebSocket dashboard or Prometheus metrics endpoint;
- add parser fuzzing tests;
- create a Linux service mode with systemd;
- add an embedded-device or hardware-sensor simulator;
- add more realistic network load scenarios;
- generate Doxygen documentation for the C++ core.

14 Interview pitch

I developed this project to demonstrate my progress in modern systems-oriented C++. The goal was to build a mini telemetry ground station able to receive UDP/TCP frames or replay files, decode them through a strict binary protocol, validate integrity with CRC, export data, measure performance and monitor operational state through degraded mode. The project primarily demonstrates my ability to structure testable C++ code, document architecture, build with CMake, verify through CI and explain technical trade-offs clearly.

15 Demo commands

15.1 Build and tests

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build -j
ctest --test-dir build --output-on-failure
```

15.2 Replay to CSV and JSON

```
./build/stgs_satellite_simulator --output-file frames.stgf --count 1000 --rate 0
./build/stgs_ground_station --replay frames.stgf --output replay.csv
./build/stgs_ground_station --replay frames.stgf --output replay.json
```

15.3 UDP simulation with loss and corruption

```
./build/stgs_ground_station --udp --port 9000 --output telemetry.json --log ground.log
./build/stgs_satellite_simulator --udp --host 127.0.0.1 --port 9000 \
  --count 5000 --rate 2000 --loss 0.02 --corrupt 0.01
```

15.4 Benchmark

```
./build/stgs_benchmark_decode --frames 200000 --payload-size 64 --decoder-threads 4
./benchmarks/run_benchmark.sh
```

16 Conclusion

C++ Space Telemetry Ground Station is a recruiter-oriented portfolio project that demonstrates solid foundations in modern C++, Linux, networking, binary protocols, multi-threading, CMake, testing, CI/CD, robustness and technical documentation. Its main value is the complete engineering approach: define a clear technical goal, design a modular architecture, implement features, test error cases, measure performance and document decisions.

For industrial, space, defense, systems or telemetry software roles, this project is a concrete proof of structured learning and of the ability to produce explainable, maintainable and verifiable code.